

1. HAL ACENTAURI HCERES POC

A reproducible bibliometric pipeline to support HCERES-style evaluation preparation.

Main message

The goal is not to automate the evaluation itself, but to generate a verifiable working basis for expert validation.

- ▶ Context: ACENTAURI scientific production indexed in HAL.
- ▶ Goal: produce deterministic tables, figures, and report artifacts.
- ▶ Constraint: keep every result auditable and rerunnable.

2. Problem Addressed

Evaluation preparation is data-heavy

HCERES-style preparation requires a reliable synthesis of publications, collaborations, venues, open-science indicators, and validation gaps.

Current bottlenecks

- ▶ HAL metadata is available but heterogeneous.
- ▶ Manual extraction is repetitive and slow.
- ▶ Inconsistencies are hard to detect afterwards.
- ▶ Missing information is often mixed with verified information.

Expected support

- ▶ Produce a first clean working basis.
- ▶ Separate computed, inferred, manual, and missing data.
- ▶ Make the validation workload explicit.
- ▶ Keep outputs reproducible.

3. Proposed Solution

Core idea

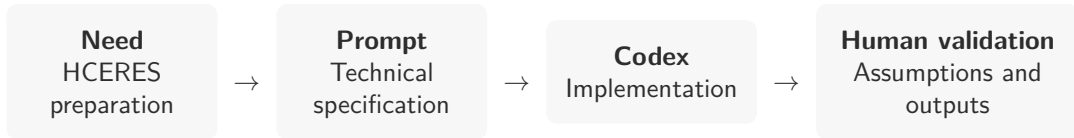
Build a local, deterministic, and rerunnable pipeline that transforms HAL metadata into HCERES-oriented indicators and report artifacts.



Design principle

Optional CSV files enrich or correct the analysis, but they are not blocking dependencies.

4. AI-Agent Development Workflow



Key message

AI is used as an accelerator inside a validation loop, not as an institutional oracle.

5. Initial Prompt to the AI

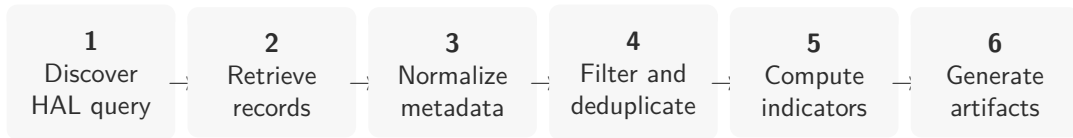
Reconstructed specification

Create a simple, monolithic Python proof of concept that automates a HAL-based bibliometric analysis of the ACENTAURI team for HCERES-style evaluation preparation.

- ▶ Query HAL automatically, with structure discovery or explicit override.
- ▶ Clean, filter, and deduplicate publication records.
- ▶ Compute indicators on output, authors, document types, collaborations, venues, and open science.
- ▶ Accept optional CSV files as enrichments without making them mandatory.
- ▶ Generate deterministic reports with explicit provenance and no invented content.

The exact prompt was not versioned; this slide summarizes the specification reflected by the generated code.

6. Implementation Pipeline



Generated artifacts

CSV tables, LaTeX tables, figures, collaboration network exports, Markdown report, and standalone LaTeX report.

Key message

The useful part is not only the final report, but the auditable path from HAL metadata to each indicator.

7. Provenance and Trust Levels

Verified
HAL metadata

Inferred
information

Manual
CSV enrichment

Missing
unavailable data

Report principle

The report exposes what is computed, inferred, manually supplied, or still requiring validation.

Why it matters

For institutional use, the value is not automation alone. The value is a transparent basis for expert validation.

8. Generated Artifacts

Repository deliverables

- ▶ Single Python module:
`hal_acentauri_hceres_poc.py`.
- ▶ CLI options for period, team, output directory, and optional CSV files.
- ▶ Static smoke tests.
- ▶ README and example CSV files.

Generated outputs

- ▶ CSV and LaTeX tables.
- ▶ Figures and collaboration network exports.
- ▶ Markdown summary report.
- ▶ Standalone LaTeX report.

Reproducibility

The pipeline can be rerun locally; outputs change only when HAL metadata or optional validation CSV files change.

9. Inside the Deterministic Report

Report logic

The report is not free-form text invented by an LLM. It is assembled from computed indicators, explicit metadata provenance, and known validation gaps.

Scope and production

- ▶ Team: ACENTAURI.
- ▶ Period: 2022–2026.
- ▶ HAL query: `structId_i:1069625`.
- ▶ Raw HAL records: 69.
- ▶ Publications retained: 58.

Audit-oriented sections

- ▶ Top authors and venues.
- ▶ Collaboration patterns.
- ▶ Open-science indicators.
- ▶ Missing optional data.
- ▶ Limitations and validation needs.

Example limitation

Theme attribution is reliable only for authors present in the theme mapping CSV or otherwise discoverable from metadata.

10. First Results Obtained

Existing run results

- ▶ 69 HAL records retrieved.
- ▶ 58 publications retained.
- ▶ 31 members identified or inferred.
- ▶ Markdown, LaTeX, and PDF reports generated.

Open-science indicators

- ▶ 56 publications with full text in HAL.
- ▶ 2 publications without full text in HAL.
- ▶ 24 publications with DOI.
- ▶ 55 English publications and 3 French publications.

Interpretation

These numbers provide a first audit basis, not a final institutional validation.

11. Demonstration Path

Demo goal

Show the complete path from execution to generated outputs, then indicate where manual validation enters the workflow.

1. Run: `python hal_acentauri_hceres_poc.py -output-dir outputs`
2. Inspect: `outputs/reports/hceres_summary_report.md`
3. Open generated LaTeX tables and figures.
4. Explain optional enrichment files: `acentauri_members.csv`, `theme_mapping.csv`, `scimago_journals.csv`, `core_conferences.csv`.

Message for the demo

The final report is useful because every indicator can be traced back to the pipeline step that produced it.

12. Narrated Video Synchronization

Goal

Make the narrated video easier to follow by aligning each slide duration with its spoken explanation.

1. Each slide has a dedicated narration paragraph.
2. The TTS voice is slowed down to improve comprehension.
3. Each slide clip duration is computed from the generated audio.
4. Long TTS silences are compressed to avoid empty waiting time.
5. A title card is inserted before the slide sequence.

AI support

Codex created the video-generation script, adjusted the voice speed, compressed silences, and regenerated the final MP4.

13. How This Video Was Generated

Generation workflow

The final MP4 was generated automatically from the Beamer PDF and the narration script.

1. Compile the Beamer source to PDF with `pdflatex`.
2. Render each PDF page as a PNG image with `pdftoppm`.
3. Generate one audio narration per slide with `edge-tts`.
4. Slow down the TTS voice to improve clarity.
5. Compress long TTS silences with `ffmpeg`.
6. Create one still-image video clip per slide.
7. Match each clip duration to its narration audio.
8. Concatenate all clips into the final MP4.

14. Limitations and Next Steps

Current limitations

- ▶ HAL metadata can be incomplete or heterogeneous.
- ▶ Member identification is partly inferred without a validated CSV.
- ▶ Theme attribution requires ARC/RIC/MOC mapping.
- ▶ Partner classification is heuristic.

Next steps

- ▶ Validate the member list.
- ▶ Provide theme mapping.
- ▶ Add Scimago and CORE reference data.
- ▶ Add domain-level quality control before institutional use.

Conclusion

AI accelerates implementation; deterministic processing makes outputs reproducible; human validation gives the result institutional value.

15. Self-Assessment and Conclusion

AI tools used

ChatGPT and Codex.

Tasks supported by AI

Idea structuring, implementation, debugging, tests, documentation, report generation, slide drafting, and video generation.

Estimated time without AI

More than one week, estimated at more than 40 hours.

Estimated time with AI

About 3 hours.

Estimated time saved

More than 37 hours, i.e. several working days.

Final message

AI accelerates implementation; deterministic processing ensures reproducibility; human validation gives institutional value.